

Unit - 5 : Function in Python

1. Introduction of Function

- Function एक block of code होता है, जो किसी specific task को perform करने के लिए बनाया जाता है।
- यह code reusability को बढ़ाता है, जिससे हमें बार-बार वही code लिखने की जरूरत नहीं होती है।
- Function की मदद से बड़े program को छोटे-छोटे parts में divide किया जा सकता है।
- Function को define करने के बाद उसे जब जरूरत होती है तब call (invoke) किया जाता है।
- Python में कई built-in functions पहले से available होते हैं, जैसे print(), len(), type() आदि।
- हम अपनी आवश्यकता के अनुसार user-defined functions भी बना सकते हैं।

2. Advantages of Using Function

- ① Code Reusability - एक बार लिखे गए code को बार-बार उपयोग किया जा सकता है।
- ② Modularity - Program को छोटे-छोटे हिस्सों में बाँटा जा सकता है।
- ③ Readability - Code को पढ़ना और समझना आसान हो जाता है।
- ④ Maintainability - किसी गलती को सुधारना आसान होता है।
- ⑤ Helps in Team Work - बड़े projects में अलग-अलग functions पर अलग-अलग लोग काम कर सकते हैं।

3. Defining Your Own Functions

- Python में अपनी खुद की function बनाने के लिए def keyword का उपयोग किया जाता है।
- Function define करने के बाद उसके नाम के साथ parentheses () लगते हैं।
- Function के अंदर जो code लिखा जाता है, उसे indented block में लिखा जाता है।
- Function कुछ value return भी कर सकता है या बिना return के केवल task perform कर सकता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    statements  
    return value # (optional)
```

Parts of a Function

1. def keyword
2. Function name
3. Parameters (optional)
4. Function body (indented code)
5. return statement (optional)

4. Example : Simple Function (Without Parameter)

```
def greet():  
    print("Hello! Welcome to Python")  
    print("Have a Nice Day!")  
  
# Function call  
greet()
```

Output :

```
Hello! Welcome to Python  
Have a Nice Day!
```

5. Example : Function with Parameter

```
def add(a, b):  
    sum = a + b  
    print("Sum is :", sum)  
  
# Function call  
add(5, 3)
```

Output :

```
Sum is : 8
```

6. Example : Function with Return Value

```
def square(n):  
    return n * n  
  
# Function call and store result  
result = square(4)  
print("Square is :", result)
```

Output :

```
Square is : 16
```

Key Points :

- ★ Function का नाम हमेशा meaningful होना चाहिए।
- ★ Python में function name case-sensitive होता है।
- ★ return statement function से value वापस भेजता है।
- ★ अगर return नहीं लिखा है, तो function None return करता है।

Parameters

- Parameter वह value होती है, जिसे Function definition के समय उसके अंदर receive किया जाता है।
- Parameter की मदद से हम Function को बाहर से data भेज सकते हैं।
- Parameter के बिना Function हर बार same result देगा, लेकिन Parameter के साथ हम अलग-अलग data पर Function को काम करवा सकते हैं।
- Python में Parameters के कई प्रकार होते हैं, जैसे Positional, Default, Keyword, Variable-length आदि।

1. Parameters क्या होते हैं ?

- Parameter एक variable की तरह होता है, जो function के अंदर value को receive करता है।
- जब हम function को call करते हैं, तब arguments के रूप में value भेजी जाती है, जो parameters में store हो जाती है।
- Parameters की मदद से function reusable और flexible बनता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    return value
```

2. Types of Parameters in Python

Python में मुख्य रूप से निम्नलिखित प्रकार के Parameters होते हैं -

- ① Positional Parameters
- ② Default Parameters
- ③ Keyword Parameters
- ④ Variable-length Parameters (*args, **kwargs)

Note :

- Parameter function definition में होता है।
- Argument function call के समय भेजी जाने वाली value होती है।
- दोनों की संख्या और order सही होना चाहिए (कुछ cases को छोड़कर)।

3. Positional Parameters

- इसमें values position के आधार पर parameters को मिलती हैं।
- function call करते समय values उसी order में भेजी जाती हैं।

Example :

```
def add(a, b):  
    sum = a + b  
    print("Sum is :", sum)  
add(5, 3)      # 5 → a, 3 → b
```

Output :

Sum is : 8

4. Default Parameters

- Default parameter वह होता है, जिसे function definition में ही default value दे दी जाती है।
- अगर function call करते समय value नहीं भेजी जाती, तो default value use होती है।

Example :

```
def greet(name = "Student"):  
    print("Hello", name)  
greet("Brajkant")      # value passed  
greet()                 # default value used
```

Output :

Hello Brajkant
Hello Student

5. Keyword Parameters

- इसमें values parameter name के साथ भेजी जाती हैं।
- इससे order की चिंता नहीं करनी पड़ती।

Example :

```
def student(name, age):  
    print("Name :", name)  
    print("Age :", age)  
student(age = 23, name = "Brajkant")
```

Output :

Name : Brajkant
Age : 23

6. Variable-length Parameters

- जब हमें function में अनिश्चित संख्या में values लेनी होती हैं, तब हम *args और **kwargs का उपयोग करते हैं।
- *args → Non-keyworded, variable-length arguments (tuple के रूप में मिलते हैं)
- **kwargs → Keyworded, variable-length arguments (dictionary के रूप में मिलते हैं)

Example :

```
def show(*args, **kwargs):  
    print("Values (args):", args)  
    print("Values (kwargs):", kwargs)  
show(1, 2, 3, name="Brajkant", age=23)
```

Output :

Values (args): (1, 2, 3)
Values (kwargs): {'name': 'Brajkant',
 'age': 23 }

Key Points :

1. Parameter function definition में लिखे जाते हैं, argument function call में दिए जाते हैं।
2. Parameters की मदद से function को reusable बनाया जा सकता है।
3. Default, keyword और variable-length parameters function को ज्यादा powerful बनाते हैं।
4. सही प्रकार का parameter चुनना program की आवश्यकता पर निर्भर करता है।

Code with Parameters, Code without Parameters in Function

1. Function without Parameters (बिना Parameters के)

- जब किसी function को define करते समय उसके अंदर कोई parameter नहीं दिया जाता, तो उसे function without parameters कहते हैं।
- यह function कुछ fixed काम करता है और उसे call करने पर वही काम execute होता है।
- इसमें बाहर से कोई value pass नहीं की जाती।
- ऐसे function का उपयोग तब किया जाता है जब function को किसी बाहरी data की आवश्यकता नहीं होती।

Syntax :

```
def function_name():  
    # function body  
    statements  
  
# function call  
function_name()
```

Example :

```
def greet():  
    print("Hello! Welcome to Python")  
    print("Have a Nice Day!")  
  
# function call  
greet()
```

Output :

Hello! Welcome to Python
Have a Nice Day!

Explanation :

- यहाँ greet() नाम का एक function बनाया गया है जिसमें कोई parameter नहीं है।
- जब हम greet() को call करते हैं, तो function के अंदर लिखे दोनों statements execute होते हैं।
- यह एक simple function है जो केवल message print करता है।

2. Function with Parameters (Parameters के साथ)

- जब function को define करते समय उसके अंदर parameter दिए जाते हैं, तो उसे function with parameters कहते हैं।
- इसमें function को call करते समय arguments के रूप में values pass की जाती हैं।
- यह function अलग-अलग values के अनुसार अलग-अलग result दे सकता है।
- यह अधिक flexible होता है और reusable code लिखने में मदद करता है।

Syntax :

```
def function_name(parameters):  
    # function body  
    statements  
  
# function call  
function_name(arguments)
```

Example :

```
def add(a,b):  
    sum = a + b  
    print("Sum is :", sum)  
  
# function call  
add(5, 3)
```

Output :

Sum is : 8

Explanation :

- यहाँ add() नाम का function बनाया गया है जिसमें दो parameters (a,b) हैं।
- जब हम add(5,3) call करते हैं, तो 5 और 3 इन parameters में pass हो जाते हैं।
- Function इन दोनों values को जोड़कर result print करता है।
- Parameter की वजह से हम एक ही function को अलग-अलग values के साथ use कर सकते हैं।

Note :

- Parameters function definition के समय लिखे जाते हैं।
- Arguments function call के समय pass की जाती हैं।
- Without parameter function fixed काम करता है।
- With parameter function dynamic होता है और अलग-अलग input ले सकता है।
- दोनों प्रकार के function Python में महत्वपूर्ण हैं और program को modular तथा reusable बनाने में मदद करते हैं।

Key Points :

- ★ Function without parameters में बाहर से कोई value नहीं दी जाती।
- ★ Function with parameters में values pass की जाती हैं।
- ★ Parameters code को reusable और flexible बनाते हैं।
- ★ दोनों का उपयोग अलग-अलग परिस्थितियों में किया जाता है।

Function Documentation

1. What is Function Documentation?

- Function documentation का मतलब है किसी function के बारे में जानकारी (description) देना, जिससे पता चले कि वह function क्या करता है, उसके parameters क्या हैं और वह क्या return करता है।
- Python में function documentation को docstring कहा जाता है।
- Docstring function की पहली line में लिखी जाती है और यह triple quotes (""" """ या ''' ''') के बीच होती है।
- यह code को समझने में मदद करती है और program को पढ़ने और maintain करने में आसान बनाती है।

2. Syntax of Function Documentation (Docstring)

```
def function_name(parameters):  
    """  
    Documentation string (Docstring)  
    """  
  
    # function body  
    return value
```

Key Points :

- ★ Docstring triple quotes में लिखी जाती है।
- ★ यह function के अंदर सबसे पहले लिखी जाती है।
- ★ यह optional होती है लेकिन अच्छी programming practice के लिए जरूरी है।
- ★ Docstring को help() या __doc__ के द्वारा देखा जा सकता है।

3. Example : Simple Function with Documentation

Code :

```
def add(a, b):  
    """  
    This function takes two numbers as parameters  
    and returns their sum.  
    """  
  
    sum = a + b  
    return sum
```

Output :

```
print(add.__doc__)
```

This function takes two numbers as parameters and returns their sum.

4. Accessing Function Documentation

- हम function की documentation को कई तरीकों से access कर सकते हैं -
- ① `function_name.__doc__` → यह docstring को string के रूप में return करता है।
- ② `help(function_name)` → यह docstring के साथ और भी जानकारी दिखाता है।
- ③ IDE (जैसे VS Code, PyCharm) में function पर hover करने से भी docstring दिखाई जाती है।

5. Example : Using __doc__ and help()

Code :

```
def greet(name):  
    """  
    This function greets the user with a message.  
    Parameter:  
        name (str): Name of the user  
    Returns:  
        str: Greeting message  
    """  
    return "Hello " + name
```

Using __doc__ :

```
print(greet.__doc__)
```

Output :

This function greets the user with a message.
Parameter:
 name (str): Name of the user
Returns:
 str: Greeting message

Using help() :

```
help(greet)
```

Output (short form) :

Help on function greet in module __main__:
greet(name)
 This function greets the user with a message.

6. Advantages of Function Documentation

- ① Code को समझने में मदद मिलती है।
- ② Team में काम करने पर दूसरों के लिए code पढ़ना आसान हो जाता है।
- ③ Program को maintain और debug करना आसान होता है।
- ④ यह professional और clean coding की पहचान है।

Conclusion :

Function documentation (docstring) Python में एक बहुत ही उपयोगी feature है। यह code की readability को बढ़ाती है, दूसरों के लिए समझने में आसान बनाती है और long-term maintenance में मदद करती है।

Keyword and Optional Parameters

1. Keyword Parameters

- Keyword parameter वह parameter होता है, जिसमें value को parameter name के साथ specify किया जाता है।
- इसमें arguments को position के अनुसार देने की restriction नहीं होती है।
- जब function में बहुत सारे parameters होते हैं, तब keyword parameter का उपयोग करने से code की readability बढ़ जाती है।
- Python में function call करते समय हम parameters को किसी भी order में उनके नाम (keyword) के साथ pass कर सकते हैं।
- यह खासकर useful होता है, जब हमें केवल कुछ specific parameters की value change करनी हो।

Syntax :

```
def function_name(param1, param2, ...):  
    # function body  
    return value  
  
# function call with keyword arguments  
function_name(param1 = value1, param2 = value2)
```

Example :

```
def student(name, age, course):  
    print("Name :", name)  
    print("Age :", age)  
    print("Course :", course)  
  
# function call (keyword arguments)  
student(age = 23, name = "Brajkant", course = "PGDCA")
```

Output :

```
Name : Brajkant  
Age : 23  
Course : PGDCA
```

Key Points :

- ★ Keyword parameter में arguments को किसी भी order में pass किया जा सकता है।
- ★ यह code को ज्यादा readable बनाता है।
- ★ सभी parameters को keyword के रूप में नहीं देना जरूरी नहीं है, कुछ normal भी हो सकते हैं।
- ★ यह तब useful होता है जब function में कई parameters हों।

2. Optional Parameters

- Optional parameter वह होता है, जिसका default value पहले से defined होता है।
- यदि function call करते समय उसकी value pass नहीं की जाती है, तो वह default value use होती है।
- Optional parameters की मदद से हम function को flexible बना सकते हैं।
- यह हमेशा keyword parameter के रूप में दिए जाते हैं (name = default_value)।
- Optional parameter का उपयोग तब होता है, जब किसी parameter की value हमेशा change नहीं होती है।

Syntax :

```
def function_name(param1, param2 = default_value):  
    # function body  
    return value  
  
# function call  
function_name(value1)           # only required argument  
function_name(value1, value2)  # with optional argument
```

Example :

```
def greet(name, msg = "Hello"):  
    print(msg, name)  
  
# function call (without optional argument)  
greet("Brajkant")  
  
# function call (with optional argument)  
greet("Brajkant", "Welcome")
```

Output :

```
Hello Brajkant  
Welcome Brajkant
```

Key Points :

- ★ Optional parameter का default value पहले से set होता है।
- ★ यदि value pass नहीं की जाती है, तो default value use होती है।
- ★ यह function को flexible और user-friendly बनाता है।
- ★ Optional parameters हमेशा keyword के रूप में लिखे जाते हैं।
- ★ इससे function call करते समय कम arguments देने पर भी function काम करता है।

Passing Collections to a Function

1. Introduction

- Collection का अर्थ होता है ऐसे data structure जिनमें एक से अधिक values store होती हैं, जैसे List, Tuple, Set, Dictionary आदि।
- Python में हम इन collections को function के अंदर argument के रूप में pass कर सकते हैं।
- जब हम collection को function में pass करते हैं, तो function उसके सभी elements पर काम कर सकता है, जैसे read करना, modify करना, add या remove करना आदि।
- List और Dictionary mutable (बदले जा सकने वाले) होते हैं, इसलिए function के अंदर उसमें किया गया परिवर्तन original collection पर भी प्रभाव डाल सकता है।

2. Passing a List to a Function

- List को function में pass करने पर हम उसके सभी elements को access कर सकते हैं।
- Function चाहे तो List को modify भी कर सकता है।

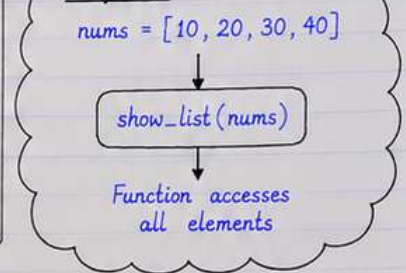
Example :

```
def show_list(items):  
    print("List elements are :")  
    for item in items:  
        print(item)  
  
nums = [10, 20, 30, 40]  
show_list(nums)
```

Output :

```
List elements are :  
10  
20  
30  
40
```

Diagram :



3. Modifying a List inside a Function

- List mutable होती है, इसलिए function के अंदर यदि हम List में changes करते हैं, तो वह original List पर भी लागू होता है।
- यह इसलिए होता है क्योंकि List को reference के द्वारा pass किया जाता है।

Example :

```
def add_item(items, value):  
    items.append(value)  
  
nums = [1, 2, 3]  
add_item(nums, 4)  
print(nums)
```

Output :

```
[1, 2, 3, 4]
```

Note :

- append() List के अंत में element जोड़ता है।
- Function के अंदर किया गया change original List को भी modify करता है।

4. Passing a Tuple to a Function

- Tuple immutable होती है, इसलिए function के अंदर उसके elements को modify नहीं कर सकते।
- हम tuple को function में pass करके उसके values को read कर सकते हैं।

Example :

```
def show_tuple(data):  
    for item in data:  
        print(item)  
  
t = (5, 10, 15)  
show_tuple(t)
```

Output :

```
5  
10  
15
```

Note :

- Tuple के elements को change करने पर error आएगा।
- Tuple केवल read-only access के लिए useful होता है।

5. Passing a Dictionary to a Function

- Dictionary को function में pass करने पर हम उसके keys और values को access कर सकते हैं।
- Function के अंदर dictionary को modify भी किया जा सकता है।

Example :

```
def show_dict(d):  
    for key, value in d.items():  
        print(key, ":", value)  
  
student = {"name": "Brajkant", "age": 23}  
show_dict(student)
```

Output :

```
name : Brajkant  
age : 23
```

Key Points :

- Collection को function में argument के रूप में pass किया जाता है।
- List और Dictionary mutable होते हैं।
- Tuple immutable होता है।
- Function के अंदर परिवर्तन original collection पर भी लागू हो सकता है।
- Collections का उपयोग करने से code अधिक modular और reusable बनता है।

Variable Number of Arguments

1. Introduction

- Python में हम किसी function को ऐसे define कर सकते हैं जो variable number of arguments ले सके, यानी function call करते समय arguments की संख्या fixed नहीं होती है।
- इसके लिए हम `*` (asterisk) operator का उपयोग करते हैं।
- यह feature तब उपयोगी होता है जब हमें पहले से यह पता न हो कि user कितने arguments pass करेगा।
- Variable number of arguments को हम दो प्रकार से use कर सकते हैं -
 - ① `*args` (Non-keyword variable arguments)
 - ② `**kwargs` (Keyword variable arguments)

2. Using `*args` (Non-keyword Arguments)

- `*args` का उपयोग तब करते हैं जब हम किसी function में variable number of positional arguments pass करना चाहते हैं।
- यह सभी arguments को एक tuple के रूप में store करता है।
- `args` केवल एक नाम है, आप इसकी जगह कोई भी valid name लिख सकते हैं।

Example :

```
def add(*args):  
    sum = 0  
    for num in args:  
        sum = sum + num  
    return sum  
  
print(add(5, 10, 15))  
print(add(1, 2, 3, 4, 5))
```

Output :

30
15

Explanation :

- यहाँ function `add()` किसी भी संख्या में arguments ले सकता है।
- सभी values `args` नाम के tuple में store होती हैं।
- for loop द्वारा सभी values को जोड़ा जाता है और result return किया जाता है।

3. Using `**kwargs` (Keyword Arguments)

- `**kwargs` का उपयोग तब करते हैं जब हम variable number of keyword arguments pass करना चाहते हैं।
- यह सभी arguments को एक dictionary के रूप में store करता है।
- `kwargs` केवल एक नाम है, आप इसकी जगह कोई भी valid name लिख सकते हैं।

Example :

```
def student_info(**kwargs):  
    for key, value in kwargs.items():  
        print(key, ":", value)  
  
student_info(name = "Brajkant",  
             age = 23, course = "PGDCA")
```

Output :

name : Brajkant
age : 23
course : PGDCA

Explanation :

- यहाँ function `student_info()` में variable number of keyword arguments pass किए गए हैं।
- सभी key-value pairs `kwargs` dictionary में store होती हैं।
- for loop द्वारा dictionary के items को access करके print किया जाता है।

4. Using Both `*args` and `**kwargs` Together

- हम एक ही function में `*args` और `**kwargs` दोनों का उपयोग कर सकते हैं।
- `*args` हमेशा `**kwargs` से पहले लिखा जाता है।
- इससे function अधिक flexible और powerful बन जाता है।

Example :

```
def demo(*args, **kwargs):  
    print("Positional arguments :", args)  
    print("Keyword arguments :", kwargs)  
  
demo(1, 2, 3, name = "Brajkant", course = "PGDCA")
```

Output :

Positional arguments : (1, 2, 3)
Keyword arguments :
{'name': 'Brajkant',
'course': 'PGDCA'}

Key Points :

- `*args` → tuple के रूप में store करता है।
- `**kwargs` → dictionary के रूप में store करता है।
- इनका उपयोग function को flexible बनाने के लिए किया जाता है।
- `*args` हमेशा `**kwargs` से पहले लिखा जाता है।
- यह feature real-time programming में बहुत उपयोगी है।

Conclusion :

Variable number of arguments का उपयोग करके हम ऐसे functions बना सकते हैं जो किसी भी संख्या में arguments को handle कर सकें, जिससे code अधिक flexible, reusable और user-friendly बनता है।

Scope • Functions - "First Class Citizens"

1. Scope

- Scope का मतलब है किसी variable का वह area जहाँ वह accessible (use) हो सकता है।
- Python में हर variable एक specific scope में defined होता है।
- Scope यह decide करता है कि हम उस variable को program के किस भाग में access कर सकते हैं।
- यदि variable अपने scope के बाहर access किया जाता है तो error आता है।
- Python में मुख्यतः तीन प्रकार के scope होते हैं - Local Scope, Global Scope और Built-in Scope।

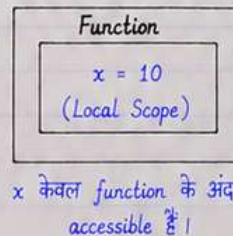
1.1 Local Scope

- जो variable किसी function के अंदर define किया जाता है, वह local scope में होता है।
- यह variable केवल उसी function के अंदर accessible होता है।
- Function के बाहर इसे access नहीं किया जा सकता।

Example :

```
def show():  
    x = 10 # local variable  
    print(x)  
  
show() # Output: 10  
print(x) # Error: x is not defined
```

Diagram : Local Scope



1.2 Global Scope

- जो variable function के बाहर define किया जाता है, वह global scope में होता है।
- यह variable पूरे program में कहीं भी access किया जा सकता है।

Example :

```
x = 5 # global variable  
def show():  
    print(x) # access global  
  
show() # Output: 5  
print(x) # Output: 5
```

1.3 Built-in Scope

- Python में कुछ pre-defined names (जैसे print(), len(), range() आदि) built-in scope में होते हैं।
- ये सभी modules और functions में automatically available होते हैं।
- इन्हें हम override नहीं करना चाहिए क्योंकि ये Python के core part होते हैं।

2. Functions - "First Class Citizens"

- Python में functions को "First Class Citizens" माना जाता है, जिसका मतलब है कि functions को भी variable की तरह treat किया जा सकता है।
- Functions को assign करना, pass करना, return करना और data structure में store करना possible है।
- यह feature Python को और अधिक flexible और powerful बनाता है।

2.1 Assigning Function to a Variable

- हम किसी function को एक variable में assign कर सकते हैं।

Example :

```
def greet():  
    print("Hello!")  
  
msg = greet # function assign  
msg() # Output: Hello!
```

2.2 Passing Function as an Argument

- हम किसी function को दूसरे function में argument के रूप में pass कर सकते हैं।

Example :

```
def greet():  
    print("Welcome!")  
  
def call_func(func):  
    func()  
  
call_func(greet) # Output: Welcome!
```

2.3 Returning a Function

- एक function दूसरे function से return भी किया जा सकता है।

Example :

```
def outer():  
    def inner():  
        print("I am inner function")  
    return inner  
  
f = outer()  
f() # Output: I am inner function
```

2.4 Storing Functions in a Data Structure

- Functions को list, tuple, dictionary आदि में store किया जा सकता है।

Example :

```
def add(a, b): return a + b  
def sub(a, b): return a - b  
  
operations = [add, sub]  
print(operations[0](5, 3)) # Output: 8  
print(operations[1](5, 3)) # Output: 2
```

Key Points :

- ★ Variable का scope यह तय करता है कि वह कहाँ accessible है।
- ★ Local variable केवल function में अंदर available होता है।
- ★ Global variable पूरे program में accessible होता है।
- ★ Built-in scope Python के pre-defined names के लिए होता है।
- ★ Python में functions "First Class Citizens" हैं।
- ★ Functions को assign, pass, return और data structure में store किया जा सकता है।
- ★ यह feature higher-order functions और functional programming के लिए बहुत उपयोगी है।

Passing Functions to a Function

1. Introduction

- Python में functions को भी objects की तरह treat किया जाता है, इसलिए हम एक function को दूसरे function के argument के रूप में pass कर सकते हैं।
- यह feature Python की "First Class Functions" property के कारण संभव होता है।
- जब हम किसी function को argument के रूप में pass करते हैं, तब हम उस function को call नहीं करते, बल्कि उसका reference पास करते हैं।
- जिस function को argument के रूप में pass किया जाता है, उसे higher-order function में कभी (parameter) के रूप में receive किया जाता है और आवश्यकता के अनुसार उसे call किया जाता है।
- यह technique code को flexible, reusable और modular बनाने में मदद करती है।

2. Syntax

```
def higher_order_function(func, other_args):  
    # func is a function passed as argument  
    func(other_args)  
  
def some_function():  
    # function body  
    pass  
  
higher_order_function(some_function, value)
```

Key Points :

- Function का नाम बिना parentheses के pass किया जाता है।
- यह function का reference पास करता है, उसका result नहीं।
- Higher-order function received function को अपने code के अंदर call करता है।
- इससे code में reusability और flexibility बढ़ती है।

3. Simple Example : Passing a Function

Code :

```
def greet():  
    print("Hello from greet()!")  
  
def call_function(func):  
    print("Calling the function...")  
    func()  
  
call_function(greet)
```

Output :

```
Calling the function...  
Hello from greet()!
```

Explanation :

- greet function को call_function में argument के रूप में pass किया गया है।
- func parameter में greet function का reference प्राप्त होता है।
- func() के द्वारा greet() function execute होता है।

4. Example with Arguments

Code :

```
def add(a, b):  
    print("Sum :", a + b)  
  
def operate(func, x, y):  
    print("Performing operation...")  
    func(x, y)  
  
operate(add, 5, 3)
```

Output :

```
Performing operation...  
Sum : 8
```

Explanation :

- यहाँ add function को operate function में argument के रूप में pass किया गया है।
- operate function, func(x, y) के द्वारा add(5, 3) को call करता है।
- इस तरह हम किसी भी function को dynamic रूप से use कर सकते हैं।

5. Real-world Use Case : Using Function as a Callback

Code :

```
def process_data(data, func):  
    result = func(data)  
    return result  
  
def square(n):  
    return n * n  
  
print(process_data(4, square))
```

Output :

```
16
```

Explanation :

- process_data function में square function को pass किया गया है।
- process_data, func(data) के द्वारा square(4) को call करता है।
- यह callback pattern का एक example है, जिसका उपयोग data processing, event handling आदि में किया जाता है।

Conclusion :

Python में function को function के argument के रूप में pass करना एक powerful feature है जो higher-order programming को संभव बनाता है और code को अधिक flexible और reusable बनाता है।

map

1. Introduction

- `map()` Python का एक built-in function है।
- यह किसी भी function को किसी iterable (जैसे List, Tuple, Set, आदि) के सभी elements पर apply करता है।
- `map()` हर element को function के अंदर भेजता है और उसका result वापस देता है।
- यह एक map object (iterator) return करता है, जिसे हम `list()`, `tuple()` आदि की help से प्राप्त कर सकते हैं।
- `map()` का उपयोग तब किया जाता है जब हमें iterable के सभी elements पर एक ही operation करना है।
- यह code को छोटा, सरल और readable बनाता है।

Syntax :

`map(function, iterable)`

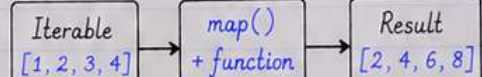
function → वह function जो
हर element पर apply होगा

iterable → List, Tuple, Set,
String आदि

2. How map() Works ?

- `map()` iterable के हर element को एक-एक करके function को देता है।
- function उस element पर operation perform करता है।
- सभी results को मिलाकर map object return होता है।
- इसे `list()` जैसी function की help से list में convert किया जा सकता है।

Working Flow :



3. Example 1 : Using map() with a User-defined Function

Code :

```
def square(n):  
    return n * n  
  
nums = [1, 2, 3, 4]  
result = map(square, nums)  
print(list(result))
```

Output :

[1, 4, 9, 16]

Explanation :

- `square()` function को `map()` के साथ pass किया गया है।
- `nums` के सभी elements पर `square()` apply होता है।
- `result` एक map object होता है, इसे list में convert करके output प्राप्त किया गया है।

4. Example 2 : Using map() with a Lambda Function

Code :

```
nums = [1, 2, 3, 4]  
result = map(lambda x: x + 10, nums)  
print(list(result))
```

Output :

[11, 12, 13, 14]

Explanation :

- यहाँ lambda function का उपयोग किया गया है।
- यह हर element में 10 add करता है।
- `map()` के साथ lambda का उपयोग code को और भी compact बनाता है।

5. Example 3 : Using map() with Multiple Iterables

Code :

```
def add(a, b):  
    return a + b  
  
list1 = [1, 2, 3]  
list2 = [10, 20, 30]  
result = map(add, list1, list2)  
print(list(result))
```

Output :

[11, 22, 33]

Explanation :

- `map()` एक से अधिक iterable को भी ले सकता है।
- यह `list1` और `list2` के corresponding elements को add करता है।
- `add()` function दोनों values को receive करके उनका sum return करता है।

6. Important Points

- `map()` हमेशा एक map object (iterator) return करता है।
- इसे `list()`, `tuple()` या `set()` की help से convert किया जा सकता है।
- यह for loop की तुलना में अधिक concise और readable होता है।
- `map()` का उपयोग किसी भी iterable (List, Tuple, Set, String, आदि) के साथ किया जा सकता है।
- यह functional programming का एक महत्वपूर्ण tool है।

Key Takeaways :

- `map()` → function + iterable
- हर element पर operation
- map object return करता है।
- `list()` से result प्राप्त करें
- code को सरल और छोटा बनाता है।

filter

1. Introduction

- `filter()` Python का एक built-in function है।
- यह किसी iterable के elements को एक condition (function) के आधार पर filter करता है।
- `filter()` हर element को function में भेजता है और केवल उन elements को return करता है जिनके लिए function True return करता है।
- यह एक filter object (iterator) return करता है, जिसे हम `list()`, `tuple()`, `set()` आदि में convert कर सकते हैं।
- `filter()` का उपयोग तब किया जाता है जब हमें iterable में से केवल कुछ specific elements को select करना होता है।
- यह code को सरल, concise और readable बनाता है।

Syntax :

`filter(function, iterable)`

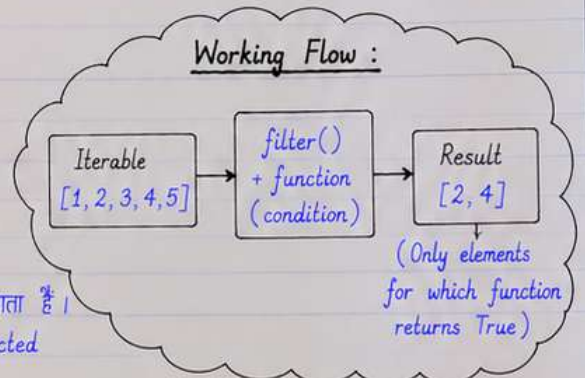
function → वह function जो हर element पर condition check करके True या False return करता है।

iterable → List, Tuple, Set, String आदि

2. How filter() Works ?

- `filter()` iterable के हर element को एक-एक करके function को भेजता है।
- function हर element पर condition check करता है।
- यदि function True return करता है, तो वह element result में शामिल होता है।
- यदि False return करता है, तो वह element skip हो जाता है।
- अंत में filter object return होता है, जिसमें केवल selected elements होते हैं।

Working Flow :



3. Example 1 : Filter Even Numbers

Code :

```
def is_even(n):  
    return n % 2 == 0  
  
nums = [1, 2, 3, 4, 5, 6]  
result = filter(is_even, nums)  
print(list(result))
```

Output :

[2, 4, 6]

Explanation :

- `is_even()` function हर number पर check करता है कि वह even है या नहीं।
- केवल वे elements जिनके लिए True return होता है, result में आते हैं।
- filter object को list में convert करके output प्राप्त किया गया है।

4. Example 2 : Filter Strings with Specific Length

Code :

```
def long_word(word):  
    return len(word) > 4  
  
words = ["cat", "python", "java", "code"]  
result = filter(long_word, words)  
print(list(result))
```

Output :

['python', 'java']

Explanation :

- `long_word()` function check करता है कि string की length 4 से अधिक है या नहीं।
- केवल वे strings जिनकी length > 4 है, result में शामिल होती हैं।
- Output में केवल "python" और "java" प्राप्त हुए हैं।

5. Example 3 : Using Lambda Function

Code :

```
nums = [10, 15, 20, 25, 30]  
result = filter(lambda x: x > 18, nums)  
print(list(result))
```

Output :

[20, 25, 30]

Explanation :

- यहाँ lambda function दिया गया है जो check करता है कि number 18 से बड़ा है या नहीं।
- जो elements condition को satisfy करते हैं, वही result में आते हैं।

6. Important Points

- `filter()` एक filter object (iterator) return करता है।
- इसे `list()`, `tuple()` या `set()` में convert किया जा सकता है।
- `filter()` का उपयोग iterable से specific elements को select करने के लिए होता है।
- यह code को छोटा, सरल और readable बनाता है।
- यह functional programming का एक महत्वपूर्ण tool है।

Key Takeaways :

- `filter()` → function + iterable
- केवल True वाले elements return होते हैं।
- Result एक filter object होता है।
- `list()` का उपयोग करके output प्राप्त करें।
- यह code को concise और efficient बनाता है।

Mapping Functions in a Dictionary

1. Introduction

- Python में हम dictionary के अंदर functions को store कर सकते हैं।
- यह technique हमें different actions (functions) को key (जैसे string) के साथ map करने की सुविधा देती है।
- इसका उपयोग तब किया जाता है जब हमें किसी key के आधार पर specific function को call करना हो।
- Dictionary में function का नाम नहीं, बल्कि function object (reference) store होता है।
- यह approach code को organized, flexible और scalable बनाता है।
- यह menu-driven program, event handling और dynamic behaviour के लिए बहुत useful होता है।

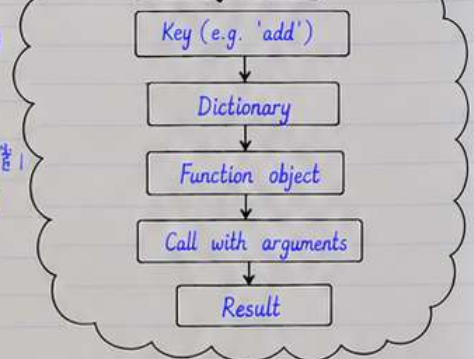
Syntax :

```
func_dict = {  
    'key1' : function1,  
    'key2' : function2,  
    ...  
}  
  
# call  
func_dict["key"](arguments)
```

2. How It Works ?

- हम functions को dictionary के key-value pair के रूप में store करते हैं।
- जब किसी key का उपयोग किया जाता है, तो उससे संबंधित function dictionary से retrieve किया जाता है।
- उस function को फिर उसके required arguments के साथ call किया जाता है।
- यदि key मौजूद नहीं है, तो KeyError आता है, इसलिए key को पहले check करना अच्छा practice होता है।

Working Flow :



3. Example 1 : Basic Function Mapping

Code :

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
  
operations = {  
    'add' : add,  
    'sub' : sub  
}  
  
print(operations['add'](5, 3))  
print(operations['sub'](5, 3))
```

Output :

8
2

Explanation :

- 'add' key के साथ add() function map किया गया है।
- 'sub' key के साथ sub() function map किया गया है।
- operations['add'] हमें add function का reference देता है, जिसे arguments के साथ call किया जाता है।
- यह approach code को dynamic और extendable बनाता है।

4. Example 2 : Using Dictionary for a Menu

Code :

```
def greet(name):  
    return f"Hello {name}!"  
  
def square(n):  
    return n * n  
  
func_dict = {  
    'greet' : greet,  
    'square' : square  
}  
  
print(func_dict['greet']("Brajant"))  
print(func_dict['square'](6))
```

Output :

Hello Brajkant!
36

Explanation :

- यहाँ 'greet' और 'square' functions dictionary में map किए गए हैं।
- Key के आधार पर सही function retrieve करके call किया गया है।
- यह menu-driven programs में बहुत useful होता है।
- इससे नए functions को आसानी से add किया जा सकता है।

5. Important Points

- Dictionary में function का reference (function object) store होता है।
- Key का type आमतौर पर string होता है, लेकिन कोई भी immutable type हो सकता है।
- Key को call करने से पहले check करना जरूरी है ताकि KeyError से बचा जा सके।
- यह technique large programs में code management को आसान बनाती है।
- Lambda functions को भी dictionary में map किया जा सकता है।
- यह method loose coupling और flexible design के लिए अच्छा माना जाता है।

Key Takeaways :

- Functions को dictionary में map किया जा सकता है।
- Key के आधार पर function call किया जाता है।
- यह approach dynamic, modular और scalable code के लिए useful है।
- Real-world applications : menu system, event handling, command mapping आदि।

Lambda

1. Introduction

- Lambda Python में एक anonymous function (बिना नाम वाला function) होता है।
- यह एक single expression में define किया जाता है।
- यह छोटे और simple functions के लिए उपयोगी होता है।
- Lambda function को lambda keyword द्वारा बनाया जाता है।
- इसका उपयोग प्रायः higher-order functions (जैसे map(), filter(), sorted()) आदि के साथ किया जाता है।
- यह code को concise और readable बनाता है।
- Lambda function का कोई नाम नहीं होता, इसलिए इसे anonymous function भी कहा जाता है।

Syntax :

lambda arguments : expression

Example :

lambda x : x * 2

- इसमें x एक argument है।
- x * 2 एक expression है।
- यह expression का result automatically return करता है।

2. Lambda vs Normal Function

Basis	Normal Function	Lambda Function
Name	Function का name होता है	कोई name नहीं (anonymous)
Syntax	def keyword का उपयोग	lambda keyword का उपयोग
Body	एक से अधिक statements हो सकते हैं	केवल एक single expression
Return	return keyword का उपयोग	Automatically return करता है
Use Case	Complex logic के लिए	छोटे और simple functions के लिए
Readability	थोड़ा बड़ा code	Compact और shorter code

3. Examples

Example 1 : Simple Lambda

Code :

```
square = lambda x : x * x  
print(square(5))
```

Output :

25

Explanation :

- यह lambda function एक number का square return करता है।
- इसे एक variable (square) में store किया गया है।

Example 2 : Lambda with Multiple Arguments

Code :

```
add = lambda x, y : x + y  
print(add(10, 20))
```

Output :

30

Explanation :

- यह दो numbers को add करता है।
- Multiple arguments भी दिए जा सकते हैं।

Example 3 : Using Lambda with Higher-order Function (map)

Code :

```
nums = [1, 2, 3, 4, 5]  
result = list(map(lambda x : x * 2, nums))  
print(result)
```

Output :

[2, 4, 6, 8, 10]

Explanation :

- map() के साथ lambda function का उपयोग किया गया है।
- यह सभी elements को 2 से multiply करता है।

4. More Uses of Lambda

- filter() के साथ → condition के आधार पर elements filter करने के लिए।
- sorted() के साथ → custom sorting के लिए।
- key parameter में → dictionary या list को specific key के आधार पर sort करने के लिए।
- Function को argument के रूप में pass करने के लिए।
- Callback functions में उपयोग।

Example : Using filter()

Code :

```
nums = [1, 2, 3, 4, 5, 6]  
even = list(filter(lambda x : x % 2 == 0, nums))  
print(even)
```

Output : [2, 4, 6]

5. Important Points

- Lambda function में केवल एक expression हो सकता है।
- यह automatically result return करता है।
- Complex logic के लिए lambda function का उपयोग नहीं करना चाहिए।
- यह code को short और clean बनाता है।
- Lambda functions functional programming में बहुत उपयोगी होते हैं।

Key Takeaways :

- Lambda → anonymous function
- Used for small and simple tasks
- Single expression only
- Works well with map(), filter(), sorted()
- Makes code concise and readable

Conclusion :

Lambda function Python की एक powerful feature है जो कम code में अधिक काम करने में मदद करती है। यह विशेष रूप से higher-order functions के साथ बहुत उपयोगी होती है।

Inner Functions, Closures

1. Introduction

- Python में किसी function के अंदर defined function को inner function (nested function) कहते हैं।
- Inner function का उपयोग code को modular और organized बनाने के लिए किया जाता है।
- Inner function, outer function के local scope को access कर सकता है।
- जब inner function, outer function के variables को remember करता है और बाद में भी use करता है, तो उसे closure कहते हैं।
- Closure हमें data को private रखने में मदद करता है।
- यह technique functional programming में बहुत useful होती है।

Key Terms :

- Inner Function → Function defined inside another function
- Closure → Function with preserved enclosing variables
- Enclosing Scope → Outer function का scope
- Nonlocal → Outer scope के variable को modify करने के लिए
- Lexical Scope → Scope determined by code structure

2. Inner Function Syntax

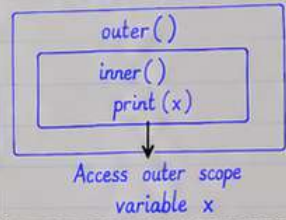
Code :

```
def outer():  
    x = 10  
    def inner():  
        print(x)  
    inner()
```

Explanation :

- inner() function, outer() के अंदर defined है।
- inner() outer function के variable x को access कर सकता है।
- यह structure encapsulation के लिए useful होता है।

Diagram :



3. Example 1 : Accessing Outer Variable

Code :

```
def outer():  
    msg = "Hello"  
    def inner():  
        print(msg)  
    inner()  
outer()
```

Output :

Hello

Explanation :

- inner() function, outer() के variable msg को access कर रहा है।
- यह closure का basic example है।
- inner() के पास अपने enclosing scope का access होता है।

4. Example 2 : Closure (Returning Inner Function)

Code :

```
def outer(a):  
    def inner(b):  
        return a + b  
    return inner  
add = outer(5)  
print(add(3))
```

Output :

8

Explanation :

- outer() एक inner function को return करता है।
- returned function (inner) अपने enclosing variable a को remember रखता है।
- यह एक closure है, क्योंकि inner() अपने definition के बाद भी variable a को access कर सकता है।

5. Example 3 : Using nonlocal Keyword

Code :

```
def counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
c = counter()  
print(c())  
print(c())
```

Output :

1
2

Explanation :

- nonlocal keyword का उपयोग outer scope के variable को modify करने के लिए किया जाता है।
- यह closure के साथ मिलकर काम करता है।
- हर बार c() call करने पर count की value update होती है, क्योंकि variable preserved रहता है।

6. Important Points

- Inner function outer function के local variables को access कर सकता है।
- Closure, function के साथ उसके enclosing scope को store करता है।
- nonlocal keyword outer scope के variable को modify करने के लिए उपयोगी है।
- यह technique encapsulation, decorators और callbacks में बहुत useful होती है।
- Closures data hiding और state maintenance के लिए powerful tool हैं।

Key Takeaways :

- Inner function → code organization
- Closure → remembers enclosing variables
- nonlocal → modify outer variable
- Useful in decorators, callbacks, etc.
- Helps in writing clean and modular code